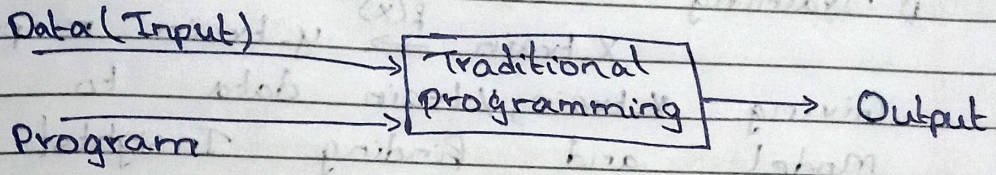
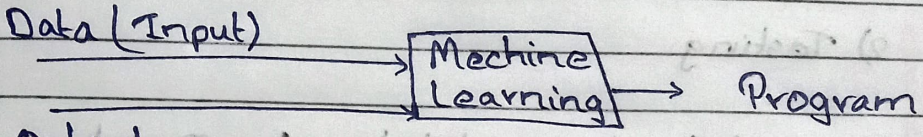
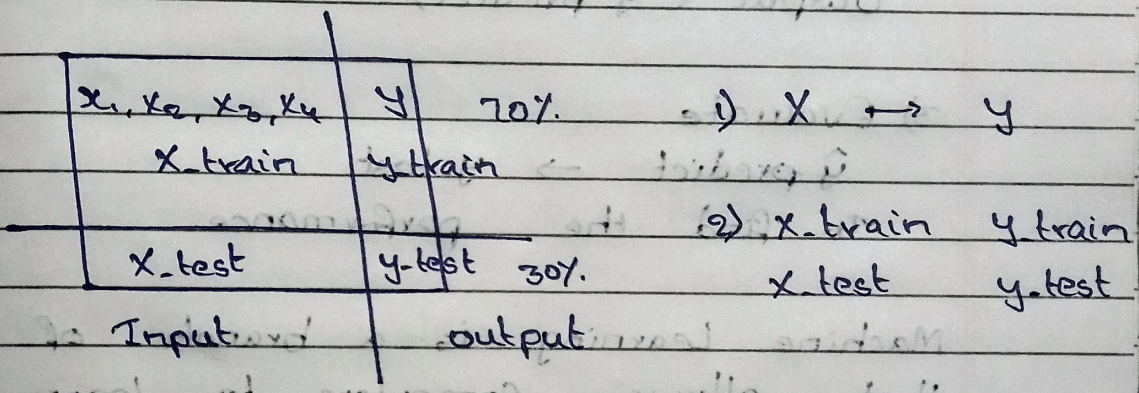
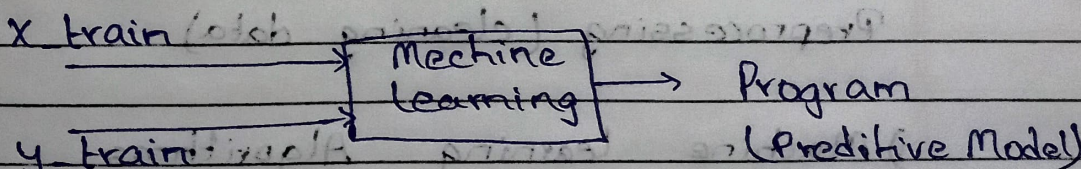
Machine Learning







→ where x_1, x_2, x_3, x_4 are input features
→ y is target value
→ Machine learn from data
1) Input (X) 2) Output (y)



→ We are mapping every input X_train Variable to output y_train Variable this process is called "Training Machine"

1) Training

$X_train \xrightarrow{f(x)} y_train$ mapping

Giving X_train, Y_train data to Model and finding Insights/ Patterns/ relationship/ hidden knowledge.

2) Testing

X_test (unknow data) to model Gives Output ($\hat{y}$ predict)

3) Evaluate

$\hat{y}$ predict → y_test

To find the performance

Machine learning is a branch of AI that allows computers to learn from data and make predictions or decisions without being explicitly Programmed for every task.

60% of Time in ML used for Preprocessing (cleaning data)

1) Machine learning Algorithms Expect Numbers
2) Machine learning Algorithms Have Requirements
3) Model Performance Depends on Data

4) Predicitive modeling is Mostly Data preparation.

Steps for Preprocessing

1) Data Imputation (Missing values)

2) Data Encoding

3) Data Normalization / scaling

4) Outlier Detection

5) Dimensionality Reduction

6) Feature selection

7) working with Imbalance data.

Data Imputation

Dealing with Missing values in excel, csv file

1) Excel file → when ever we find Blank Data referred as ☐ → Blank, NaN → Not a Number, we must fill missing values with df.fillna()

→ when data is Numerical / Quantitative data use Mean / Median
when no outlier ↙ : when outlier present in data ↘ in data

→ when data is Categorical / Qualitative data use Mode [0]

→ Can also fill with Constant value as per domain knowledge

2) Instead of blank data we have '0' Zero

3) Some times we have special character '?'

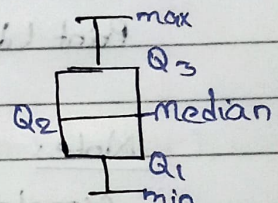
Using Machine Learning Library
Scikit - learn (sklearn)

	steps
Simple Imputer KNN Imputer Iterative Imputer	1) Import respecte library 2) Create an object of class 3) call fit_transform(data) using object.

Special Case Impute

→ we can write our own function for Impute.

Example: Boxplot only for Numerical data



Note:
How to find out missing values in df.

```
df.isna().sum()
df.isnull().sum()
df.count()
df.info()
```

Simple Imputer

```
sklearn.impute.SimpleImputer(strategy='',
                             fill_value=)
df.columns
```

```
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = df[col].fillna(df[col].mode()[0])
    else:
        df[col] = df[col].fillna(df[col].median())
```

```
From sklearn.impute import SimpleImputer
num_imputer = SimpleImuter (strategy = "median")
df[['Age', 'Salary']] = num_imputer.fit_transform
                        (df[['Age', 'Salary']])
Print (df)
```

Note:

SimpleImputer (strategy = "mean/median/
most_frequenay / constant")

mean / median for numerical
most_frequent for categorical
Constant for fill_value

KNNImputer

KNNImputer works only with numerical columns.

```
From sklearn.impuler import KNNImputer
imputer = KNNImputer (n_neighbors = 2)
df['Age'] = imputer.fit_transform(df['Age'])
```

Data Encoding

→ Machine Learning Algorithm does not understant Categorical data.
→ So we need to Convert Categorical data to Numerical data is know as Data Encoding.

1) map()
2) Sklearn LableEncoder
3) get_dummies()
4) Sklearn OneHotEncoder
5) Sklearn OrdinalEncoder

map()

df['workclass'].unique.
→ array(['Private', 'state-gov', 'central-gov', 'other'], dtype='object')
workclass_mapping = {'Private':0, 'state-gov':1, 'central-gov':2, 'other':3}
df['workclass'] = df['workclass'].map(workclass_mapping)

Sklearn LableEncoder

when dividing data into input X and output y
X is a 2D dataFram
but y is 1D series
but sklearn expect y also as 2D
Using LableEncoder we are Converting 1D to 2D
→ LableEncoder Converts text Categories into numbers.
→ Use it for binary categories (Yes/No, Male/Female)

```
from sklearn.preprocessing import
LableEncoder
le = LableEncoder()
df['workclass'] = le.fit_transform(df['workclass'])
```

get_dummies()

Create a separate column for each category.

```
df = pd.get_dummies(df, columns=['workclass'])
print(df)
```

→ For integer data type
add dtype = int

this is dis-advantage is, it is going to create more number of new columns deping on the class lables. Create sparse Matrix

Sklearn OneHotEncoder

for Nominal data

No order in the data

Example:

- One
- Two
- three

- Python
- ML
- DL
- AI

```
from sklearn.preprocessing import
OneHotEncoder
encoder = OneHotEncoder(sparse_output=False)
encoded = encoder.fit_transform(df[['Deparment'
```

encoded_df = pd.DataFrame(encode, columns = encoder.get_feature_names_out(['Department']))

Sklearn OrdinalEncoder

Contains Inherent order in the data

1. Tenth	A. Small
2. Inter	B. Medium
3. Graduate	C. Large
4. Post Graduate	D. Extra Large
5. PhD	

```
from sklearn.preprocessing import OrdinalEncoder
encoder = OrdinalEncoder(categories=[['school', 'college', 'master', 'PhD']])
df['Eduction'] = encoder.fit_transform(df[['Eduction']])
```

Note:

To remove unwanted spaces

```
for col in df.columns:
    df[col] = df[col].str.strip()
```

Data Normalization

why Data Normalization ?

→ few features in the dataset, may influence other features.

→ A feature which has large value, may influence other feature which has small value.

→ so we need to make all features with same Range is know as Data Normalization.

what is advantage ?

Most of ML algorithms work will, when data is Normalized.

Techniques

1) Min-Max :

Min-max is a scaling technique where values are rescaled and shifted so that they range between 0 and 1 or between -1 and 1.

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

2) ~~Z-score~~

```
from sklearn.preprocessing import
MinMaxScaler
Scaler = MinMaxScaler()
```

2) Z-score

$$\frac{Xi - \mu}{\sigma}$$

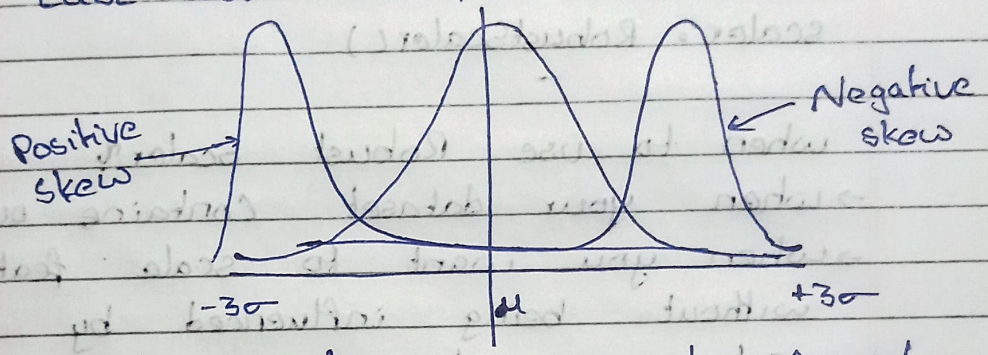
where μ is zero

σ is -3 to +3 (-1 to 1)

```
from sklearn.preprocessing import
standardScaler.
scaler = standardScaler()
```

3) Transformation

Log Transformation
Square root Transformation } skewness
Cube root Transformation



To reduce skewness or to convert data into Normalize use log, square root, cube root

Log Transformation:
→ Can't be applyed to zero or negative values.
→ It is generally used for reducing right skewness of variables.

Square root Transformation:
square root can be applied to positive values including zero.

Cube root Transformation:
Can be applied to negative values including zero.

4) Robust Scalar

Can be used when more outliers present in data.

$$\frac{x_i - \text{Median}}{IQR}$$

```
from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
```

When to use Robust Scaler?

→ When your dataset contains outliers
→ When you want to scale features without being influenced by extreme values.
→ When the data is not normally distributed.

5) Power Transformer

Yeo-Johnson Transformer:
works for both positive and negative values.

Box-Cox Transformer:
works only for positive values.

```
from sklearn.preprocessing import PowerTransformer
pt = PowerTransformer(method="yeo-johnson")
        or (method="box-cox")
```

When to use Power Transform ?

When your data is skewed and you want to make it more Gaussian-like

Feature Selection

Data (Features)

Input data

- → Non Target columns
- → Predictor Var
- → Independent Var

Output data

- → Target columns
- → Response Var
- → dependent Var

Stored Data

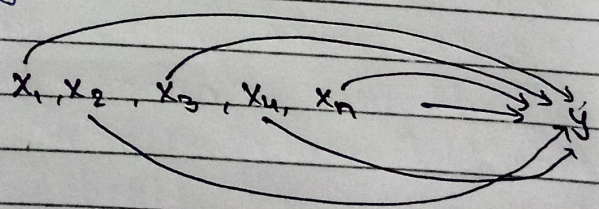
observations, Data Points, Instances.

Every column is one feature

To Build the Model in ML for problem.

we need to identify what independent features (predictor var are important for model building is called as Feature selection.

→ we are going to select Important feature for model building to increase Performance of ML algorithms.



mean:

Finding Relevent Feature for Model Building for Improvement of performance of ML Model

→ Feature selection: is a crucial step in machine learning to improve model performance, reduce overfitting and enhance interpretability

→ Feature selection: involves selecing the most relevant features from the dataset while discarding irrelevant or redundant ones.

1) Filter Methods
Uses statistical Techiques (chi-square, correlation, mutal information, Anova)
→ gives quantifiable score for every feature
→ sort in descending order
Pick top N features based on score

sklearn.feature_selection.SelectionKBest(
score_func = <funtion>, k)
where score_func = any statistical Techiques
k = top k Number.

chi2 (chi-square test)
Type: Supervised, for classification
Method: Measure dependency between categorical feature and target using chi-squared statistics.
Requirment: feature must be non-negative (0 or positive)
Props: Good for categorical

```
from sklearn.feature_selection import
    SelectKBest, chi2
selector = SelectKBest(score_func = chi2, k=5)
x_new = selector.fit_transform(X,y)
```

Mutual_info_classif (Mutal information)

Type: Supervised, for classification

Method: Measure non-linear dependency between each feature and target.

Requirement: Works for Continous or discrete feature.

Pros: Can detect complex non-linear relationships that F-test cannot.

```
from sklearn.feature_selection import
    SelectKBest, mutal_info_classif.
selector = SelectKBest(score_func = mutal_info
_classif, k=5)
x_new = select.fit_transform(X,y)
```

F_classif (ANOVA F-values)

Type: Supervised, for classification

Method: Measure linear dependency between each feature and the target using ANOVA F-test

Requirement: Features should be Continous (numeric), target should be categorical

Props: Fast, interpretable, stand for many datasets.

```
from sklearn.feature_selection import
    selectKBest, f_classif
selector = selectKBest(score_func=f_classif, k=5)
X_new = selector.fit_transform(X,Y)
```

Function	Feature Type	Target Type	Captures linear
chi2	Non-negative	Categorical	linear
mutual_info_classif	Continous or discrete	"	linear
f_classif	Continous	"	Non-linear

fit()

Generally fit() used to Training or mapping, Input Variable to Output Variable.

selector.get_support() :- tells us which features were selected

selector.scores_ :- tells us scores of features

SelectPercentile

→ It select features based on their statistical score.

→ Instead k features, you choose a percentile of features.

Ex:- top 20% features.

```
from sklearn.feature_selection import
    SelectPercentile, chi2
selector = selectPercentile(chi2, percentile=75)
```

2) Wrapper Methods

are iterative methods for select best feature.

1. Forward selection:

start with zero features $\{\} \rightarrow \{y\}$

for every iteration we add few features

$\{x_1, x_2\} \rightarrow \{y\}$

adding more features iteratively for input data.

$\{x_1, x_2, x_3, \ldots x_n\} \rightarrow \{y\}$

whereever we get best performance we stop, faster

2. Backward Elimination:

we start with all features

$\{x_1, x_2, x_3, x_4, x_n\} \rightarrow \{y\}$

in every iteration, we will eliminate un important features

$\{x_1, x_2, x_3, \ldots x_{n-1}\} \rightarrow \{y\}$

finally we stop with zero features

Forward selection	Backward Selection
start with 0 features	start with all features
Add best feature	Removes worst feature
Faster for many features	Better when data small
Good for large dataset	More computationally expensive

Sequential Feature Selector

```
from sklearn.feature_selection import
SequentialFeatureSelector
selector = SequentialFeatureSelection(model,
n_features_to_select = 10, direction = "forward",
scoreing="r2", CV = 5)
```

Same for backward.

Recursive Feature Elimination

Recursively removes the least important features based on model weights.

```
from sklearn.feature_selection import RFE
rfe = rfe(model, n_features_to_select = 10)
```

Wrapper methods evaluate subset of features by training and tesing a model iteratively.

3) Embedded Methos

perform feature selection as part of the Model training process.

3.1) we will create a ML model and pass ML model as argument for selectFromModel(model)

3.2) In ML we have various kinds of ML models. such as Linear Models. Use model.coef[0]

LinearRegression, Lasso, Ridge, Logistic

Tree Model etc

Uses model.feature_important.

Decision Tree, Random Forest, XGBoost

1. Train Model (like SVM, Logistic, Random)
2. The model assign important to each feature
   * Coefficient (coef_) → linear model
   * Feature importance (feature_importances_)
     ⇒ Tree model
3. SelectFromModel:
   * Remove features with low importance
   * Keeps features with high importance

```
from sklearn.feature_selection import SelectFromModel
for linear  Print(model.coef_)
for Tree    Print(model.feature_importances_)
selector = SelectFromModel(model, threshold=0.05)
```

Dimensionality Reduction

→ when $x_1, x_2, x_3, \ldots x_n$ features are increasing leads to overfit

Overfit: For ML model with Training data gives good performance with Testing data gives poor performance.

→ So we need to remove redundent, duplicate features.

Techniques

1. Principle Component Analysis (PCA)
2. Single Value Decomposition (SVD)
3. Linear Discrimination Analysis (LDA)

1. & 2 are un-supervised learning (ND target Var required)

3 is supervised learing (both X and Y needed)

above 3 techniques are called Matrix Factorization Techniques

```
from sklearn.decomposition import PCA
from sklearn.decomposition import
TruncatedSVD
```

★ Explainable AI

SHAP : It is a popular Explainable AI (XAI) technique used to understand why a machine-learing model made a particular prediction.

```
Pip install shap
import shap
explainer = shap.TreeExplainer(model)
shap.summary_plot
shap.waterfall_plot
```